

Chapitre 1 – Le langage C pour les systèmes embarqués

volatile

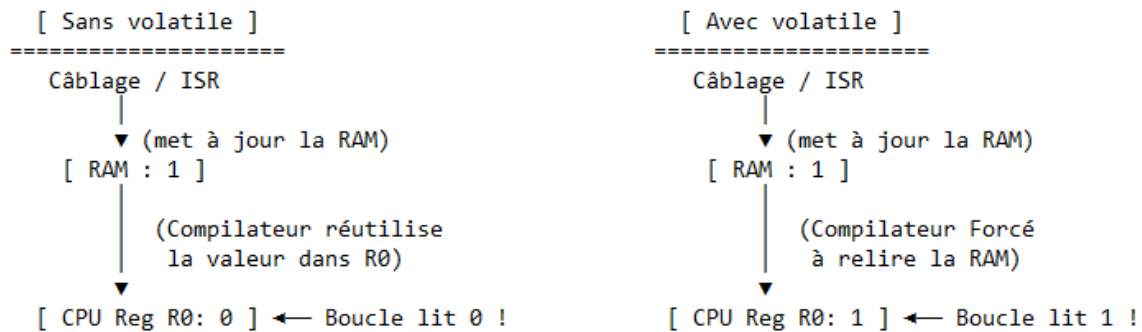


Schéma explicatif : Avec vs Sans volatile

1. Définition

Le mot-clé volatile indique au compilateur qu'une variable peut changer de valeur à tout moment **en dehors du flux normal d'exécution du programme**, sans que le code courant ne la modifie directement.

Principe d'or : volatile désactive l'optimisation du compilateur pour la variable ciblée. Il force le processeur à effectuer une **lecture/écriture physique en mémoire RAM ou registre (MMIO)** à chaque accès, au lieu d'utiliser une copie temporaire mise en cache dans un registre interne du CPU.

2. Pourquoi est-ce important ?

En systèmes embarqués, les variables peuvent être modifiées par :

- Une interruption (**ISR** – *Interrupt Service Routine*)
- Un périphérique matériel (**MMIO** / Registre I/O)
- Une autre tâche dans un **RTOS**

Sans **volatile**, le compilateur applique des optimisations : il stocke la valeur dans un registre CPU interne et réutilise cette ancienne valeur sans relire la mémoire RAM.

3. STM32 (ARM Cortex-M — ARM-GCC)

```
#include "stm32f4xx_hal.h"

// Variable modifiée dans l'ISR et lue dans la boucle principale
static volatile uint8_t button_pressed = 0;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {

    if (GPIO_Pin == BUTTON_PIN) {

        button_pressed = 1; // Modifié par l'interruption

    }

}

int main(void) {

    HAL_Init();

    while (1) {

        if (button_pressed) { // Relecture forcée de la RAM

            button_pressed = 0;

            // Traiter l'appui

        }

    }

}
```

4. PIC32 (MIPS32 — MPLAB XC32)

```
#include <xc.h>

#include <sys/attribs.h>

static volatile uint32_t timer_ticks = 0;

void __ISR(_TIMER_1_VECTOR, IPL2AUTO) Timer1Handler(void) {

    timer_ticks++;

    IFS0bits.T1IF = 0; // Clear flag

}

void delay_ms(uint32_t ms) {

    uint32_t start_ticks = timer_ticks;
```

```

    while ((timer_ticks - start_ticks) < ms); // Forcer la relecture de
timer_ticks
}

```

5. SAMD21 / Atmel (ARM Cortex-M0+ — GCC)

```

#include <sam.h>

static volatile uint8_t rx_data_ready = 0;

static volatile uint8_t rx_byte = 0;

void SERCOM0_Handler(void) {

    if (SERCOM0->USART.INTFLAG.bit.RXC) {

        rx_byte = SERCOM0->USART.DATA.reg;

        rx_data_ready = 1;

    }

}

```

6. Microcontrôleur 8051 (8 bits — Keil C51 / SDCC)

```

#include <8051.h>

static volatile __bit flag_10ms = 0;

void timer0_isr(void) __interrupt(1) {

    flag_10ms = 1;

}

void main(void) {

    while (1) {

        if (flag_10ms) {

            flag_10ms = 0;

            // Traitement

        }

    }

}

```

7. Utilisation dans Visual Studio Code

Dans **VS Code**, volatile est pris en charge nativement via **IntelliSense** ou **Clangd**. Il permet d'éviter les fausses erreurs d'analyse statique lors de l'utilisation de chaînes de compilation comme arm-none-eabi-gcc, xc32-gcc ou sdcc (via PlatformIO ou EIDE).

8. Question d'entrevue

Pourquoi utilisez-vous le mot-clé 'volatile' ?

Réponse professionnelle

J'utilise 'volatile' lorsqu'une variable peut être modifiée en dehors du flux normal d'exécution, par exemple dans une interruption, un registre matériel ou une tâche RTOS. Cela garantit que le compilateur relit toujours la valeur en mémoire.

Avantages

- Empêche certaines optimisations dangereuses.
- Essentiel pour les registres matériels et les ISR.
- Améliore la fiabilité du firmware.

Limites

- 'volatile' ne rend pas une variable atomique.
- Il ne remplace pas un mutex ni une section critique.

À retenir

Ne pas utiliser 'volatile' partout. L'utiliser uniquement lorsque la variable peut être modifiée par le matériel, une interruption ou une autre tâche.

pointeur

1. Qu'est-ce qu'un pointeur ?

Un pointeur est une variable qui contient l'adresse mémoire d'une autre variable.

2. Pourquoi les pointeurs sont-ils importants ?

Ils permettent de manipuler efficacement la mémoire, de passer des paramètres sans copie et d'accéder aux périphériques matériels via des registres.

3. Exemple simple

```
int value = 10;  
int *ptr = &value;  
*ptr = 20;
```

4. Exemple STM32

Les registres des périphériques sont accessibles par des adresses mémoire. Les bibliothèques STM32 utilisent largement les pointeurs pour manipuler les registres et les structures HAL.

Avantages

- Performances
- Faible consommation mémoire
- Manipulation directe du matériel
- Structures de données complexes

Inconvénients

- Pointeurs invalides
- Déréférencement NULL
- Dépassements mémoire
- Débogage plus difficile

5. Quand utiliser ?

Pour les drivers, buffers DMA, registres matériels, grandes structures ou fonctions manipulant des données volumineuses.

6. Quand éviter ?

Pour des variables simples lorsqu'une copie est suffisante ou lorsque le code deviendrait difficile à maintenir.

7. Questions d'entrevue

- Quelle est la différence entre un pointeur et une variable ?
- Quelle est la différence entre NULL et un pointeur non initialisé ?
- Que se passe-t-il si on déréférence un pointeur NULL ?

Réponse professionnelle

Les pointeurs sont indispensables en développement embarqué car ils permettent un accès direct à la mémoire et aux périphériques. Cependant, ils doivent être utilisés avec prudence afin d'éviter les erreurs mémoire et les comportements indéfinis.

À retenir

Un bon ingénieur Firmware ne sait pas seulement utiliser les pointeurs ; il sait quand les utiliser et comment garantir la sécurité et la lisibilité du code.

const

1. Définition

Le mot-clé const indique qu'une donnée ne doit pas être modifiée après son initialisation.

2. Pourquoi est-ce important ?

Utiliser const pour protéger les données, améliorer la lisibilité et permettre certaines optimisations.

3. Exemple

Exemple : `const uint16_t ADC_MAX = 4095;`

4. Question d'entrevue

Question : Quelle est la différence entre const et #define ?

Réponse professionnelle

Réponse : const possède un type, est vérifié par le compilateur et peut être placé en mémoire Flash.

À retenir

N'utiliser const que lorsque cela répond à un besoin clair de conception.

static

1. Définition

Le mot-clé static modifie la **durée de vie** ou la **visibilité** d'une variable ou d'une fonction.

Important pour l'entretien : Le fonctionnement de static est **100 % universel et identique** sur toutes les architectures 32 bits (**STM32** / ARM Cortex-M, **PIC32** / MIPS32, **AVR32** / **Atmel SAM** / UC3). Le comportement est défini par la norme du langage C (ANSI C / C99 / C11) et géré par le compilateur (GCC, XC32, IAR), indépendamment du matériel.

2. Représentation visuelle : Organisation de la mémoire RAM (SRAM 32 bits)

Peut importe la marque du microcontrôleur (STM32, PIC32 ou AVR32), le compilateur alloue la mémoire RAM selon la même structure. Les variables static sont placées dans

Organisation Mémoire SRAM (Architectures 32-bits)

Adresse Basse (ex: 0x20000000)	
Section .data	Variables 'static' & globales initialisées (<code>!= 0</code>) (Valeurs initiales copiées depuis la Flash au Boot)
Section .bss	Variables 'static' & globales non initialisées ou <code>= 0</code> (Mises à zéro automatiquement par <code>crt0.s</code> au Reset)
Heap (Tas) ... Stack (Pile)	Dynamic memory (<code>malloc/free</code>) → Grandit vers le haut Variables locales automatiques ← Grandit vers le bas (Géré par le Pointeur de Pile / Stack Pointer SP)
Adresse Haute (ex: 0x20007FFF)	

la mémoire permanente (`.data` / `.bss`) et **jamais dans la pile (*Stack*)**.

*Schéma : Implantation en mémoire SRAM d'une variable static (`.data` / `.bss`) opposée à une variable locale standard (*Stack*).*

3. Exemple STM32 (ARM Cortex-M0+ / M4 — Compilateur ARM-GCC)

Gestion d'un bouton poussoir avec anti-rebond dans un callback GPIO :

```

#include "stm32f4xx_hal.h"

// 1. VARIABLE GLOBALE STATIC : Privée au fichier stm32_driver.c
static volatile uint32_t system_ticks = 0;

// 2. FONCTION STATIC : Inaccessible depuis d'autres fichiers .c
static void reset_timer_hardware(void) {
    __HAL_TIM_SET_COUNTER(&htim2, 0);
}

// Callback d'interruption GPIO
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == BUTTON_PIN) {
        // 3. VARIABLE LOCALE STATIC : Stockée en RAM (.bss), conserve sa valeur
        static uint32_t last_interrupt_time = 0;

        uint32_t current_time = HAL_GetTick();

        // Filtre anti-rebond (Debounce) de 200 ms
        if (current_time - last_interrupt_time > 200) {
            reset_timer_hardware();

            last_interrupt_time = current_time; // Conservé pour le prochain appel
        }
    }
}

```

4. Exemple PIC32 (MIPS32 — Compilateur MPLAB XC32)

Gestion du timer système Timer1 sur PIC32MX / PIC32MZ :

```

#include <xc.h>

#include <sys/attribs.h>

// 1. VARIABLE GLOBALE STATIC : Masquée du Linker (Encapsulation)
static volatile uint32_t milli_seconds = 0;

// 2. FONCTION STATIC : Helper privé au driver
static void pic32_clear_timer_flag(void) {
    IFS0bits.T1IF = 0; // Clear Flag Timer1 sur PIC32
}

```

```

// Routine de service d'interruption (ISR) Timer1

void __ISR(_TIMER_1_VECTOR, IPL2AUTO) Timer1Handler(void) {

    // 3. VARIABLE LOCALE STATIC : Conserve l'état du compteur secondaire en RAM

    static uint16_t prescaler_counter = 0;

    milli_seconds++;

    prescaler_counter++;

    if (prescaler_counter >= 1000) {

        // Action chaque seconde

        prescaler_counter = 0;

    }

    pic32_clear_timer_flag();

}

```

5. Exemple AVR32 / Atmel (Series AT32UC3 / ATSAMD21 — GCC)

Driver pour composant Atmel / Microchip 32 bits (ex: ATSAMD21 / AT32UC3) :

```

#include <sam.h> // Ou <avr32/io.h> selon la gamme AT32 / SAM

// 1. VARIABLE GLOBALE STATIC : Interdite d'accès via 'extern' dans un autre
module

static volatile uint8_t usart_tx_busy = 0;

// 2. FONCTION STATIC : Interne à la couche abstraction matérielle (HAL)

static inline void atmel_usart_enable_tx(Usart *usart) {

    usart->US_CR = US_CR_TXEN;

}

// Handler d'interruption USART sur Atmel 32 bits

void USART0_Handler(void) {

    // 3. VARIABLE LOCALE STATIC : Maintient l'index du tampon de transmission

    static uint16_t tx_buffer_index = 0;

    if (USART0->US_CSR & US_CSR_TXRDY) {

```

```

    // Envoi du caractère suivant...

    tx_buffer_index++;

}}

```

6. Tableau récapitulatif

Portée de static	Emplacement mémoire	Visibilité (Linkage)	Objectif principal
Variable locale	RAM (.data / .bss)	Interne à la fonction	Conserver un état/compteur sans créer de variable globale.
Variable globale	RAM (.data / .bss)	Interne au fichier .c	Isoler les variables internes d'un driver (Encapsulation).
Fonction	Flash (.text)	Interne au fichier .c	Créer des fonctions "helpers" privées et éviter les conflits de nom au linkage.

7. Question d'entrevue

Question 1 : Pourquoi utiliser static ?

Réponse professionnelle

Réponse : Pour limiter la portée d'une fonction ou conserver un état interne sans utiliser une variable globale.

À retenir: N'utiliser static que lorsque cela répond à un besoin clair de conception.

Question 2 : "Est-ce que le comportement de static change selon l'architecture (ARM, MIPS, AVR32) ?"

Réponse professionnelle :

"Non. static est un mot-clé défini par la norme du langage C. Que ce soit sur un STM32 (ARM Cortex-M), un PIC32 (MIPS) ou un Atmel AVR32, le compilateur applique les mêmes règles : l'allocation se fait dans les sections RAM fixes (.data

ou .bss) au lieu de la pile (Stack), et la visibilité des symboles globaux est restreinte au seul fichier objet (.o) courant."

Question 3 : "Pourquoi préférer une variable locale static à une variable globale ?"

Réponse professionnelle :

"Pour respecter le principe du moindre privilège (encapsulation). Une variable locale static conserve sa valeur entre les appels de fonction tout en restant strictement inaccessible depuis l'extérieur de cette fonction. Cela évite les effets de bord indésirables et sécurise le code dans les architectures embarquées complexes."

Question 4: Peut-on écrire ce type de code et utiliser static dans Visual Studio Code ?

Réponse professionnelle :

Oui, absolument !

Visual Studio Code (VS Code) est un éditeur de code universel. Il est totalement indépendant de l'architecture matérielle ciblée par votre code C (8051, ARM Cortex-M, PIC32, AVR, ESP32 ou x86 Linux).

Dans VS Code, les mots-clés static, volatile et extern sont pris en charge nativement grâce aux extensions de coloration syntaxique et d'analyse de code C/C++

une question orientée

Question 5(option): Comment les développeurs utilisent VS Code pour les systèmes embarqués?

Réponse professionnelle :

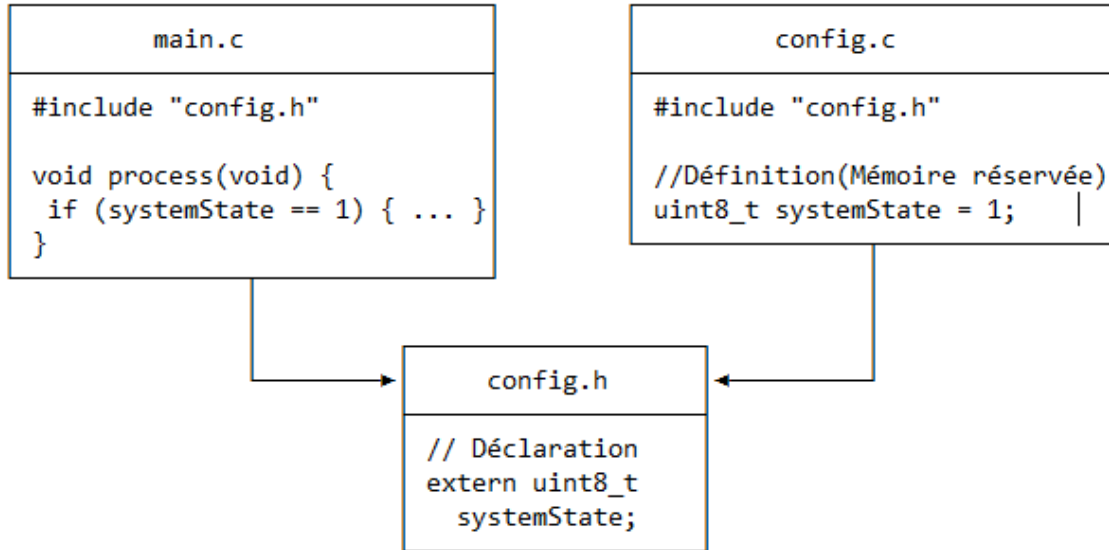
- **Pour ARM Cortex-M / STM32 / SAMD21** : Utilisation de VS Code avec l'extension **PlatformIO**, **Cortex-Debug** ou l'extension officielle **STM32 VS Code Extension**, couplée à la chaîne de compilation arm-none-eabi-gcc.
- **Pour 8051 (8bits MCU)** : Utilisation de VS Code avec l'extension **EIDE** (*Embedded IDE*) ou **PlatformIO**, combinée avec le compilateur open-source **SDCC** (*Small Device C Compiler*).
- **Pour PIC32** : VS Code se configure pour fonctionner directement avec la chaîne de compilation **MPLAB XC32**.

Question	Réponse
Le comportement de static est-il universel en C ?	Oui , des architectures 8 bits (8051, AVR) aux architectures 32 bits (ARM, PIC32) et 64 bits.
Quelle est l'architecture du 8051 ?	Il s'agit d'une architecture 8 bits .
VS Code est-il adapté à l'embarqué ?	Oui , c'est aujourd'hui l'un des standards majeurs de l'industrie pour l'écriture de firmwares en C/C++.

Extern

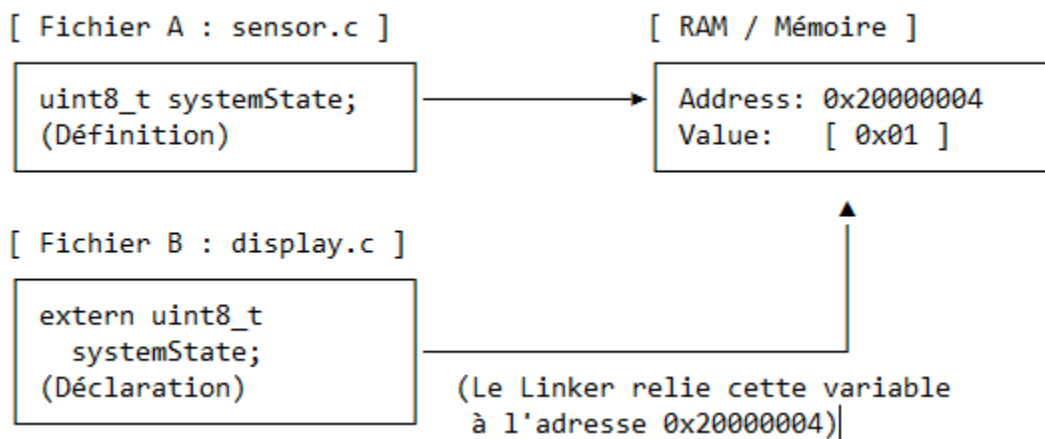
Option 1 : Schéma ASCII / Diagramme de structure (Prêt à copier-coller)

Ce type de diagramme montre exactement la différence entre **Définition** (allocation mémoire) et **Déclaration** (référence) :



Option 2 : Schéma conceptuel Mémoire & Editeur de liens (Linker)

Vous pouvez aussi représenter le fonctionnement sous forme de bloc mémoire :



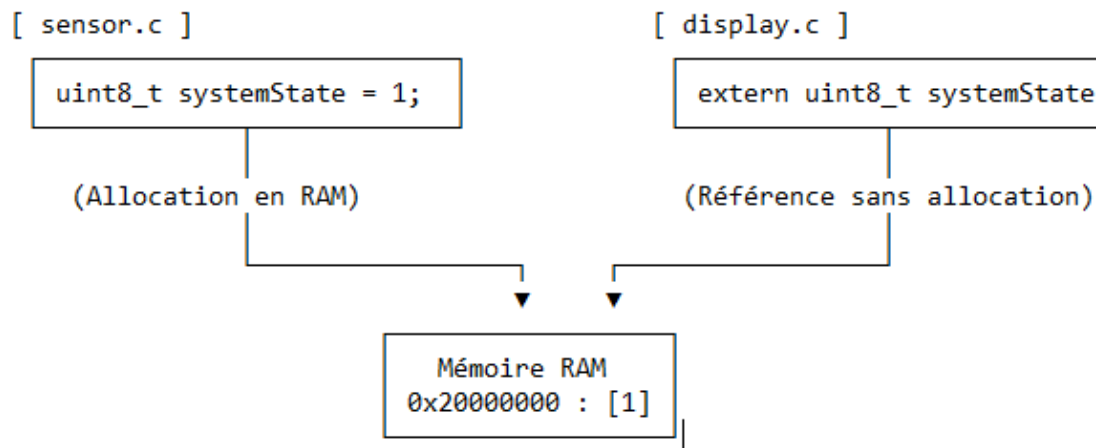
1. Définition

Le mot-clé **extern** permet de **déclarer** l'existence d'une variable ou d'une fonction globale sans lui allouer de mémoire dans le fichier courant. Il indique au compilateur et à l'éditeur de liens (*linker*) que la variable est définie dans un autre fichier `.c`.

2. Pourquoi est-ce important ?

Il évite les redéfinitions multiples lors de la compilation (*multiply defined symbols*) et permet de partager des états globaux entre différents modules logiciels tout en gardant un projet bien structuré.

Représentation visuelle : Déclaration vs Définition



3. Exemple

Fichier `config.c` (Définition) :

```
#include <stdint.h>

// Définition : La mémoire est physiquement allouée ici

uint8_t systemState = 0;
```

Fichier `config.h` (Déclaration partagée) :

```
#ifndef CONFIG_H
#define CONFIG_H

#include <stdint.h>
```

```
// Déclaration : Indique que systemState existe ailleurs
```

```
extern uint8_t systemState;
```

```
#endif
```

Fichier main.c (Utilisation) :

```
#include "config.h"
```

```
void loop(void) {
```

```
    if (systemState == 1) {
```

```
        // Traitement
```

```
    }
```

```
}
```

4. Question d'entrevue

Question : Quelle est la différence entre une déclaration et une définition ?

Réponse professionnelle

"Une **déclaration** (avec `extern`) indique au compilateur le nom et le type d'une variable sans lui allouer d'espace mémoire. Une **définition** réserve l'espace mémoire physique pour la variable. On peut déclarer une variable plusieurs fois dans différents fichiers, mais on ne doit la définir qu'une seule fois."

À retenir

- Ne pas abuser des variables globales partagées avec `extern` : privilégier l'encapsulation (fonctions d'accès / getters-setters et mot-clé `static`).
- N'utiliser `extern` que lorsque cela répond à un besoin clair de conception inter-modules.

